

Eindhoven Mood Radar — ML Model Training

Fontys ICT | Moonshot Group | AI for Society Semester

What does this notebook do?

This notebook teaches a computer to recognise the **mood of a city zone** in Eindhoven.

We give it a list of historical examples — things like *"it was a Saturday night, there were 3 music events, PSV was playing, and it was 14 degrees"* — and we tell it what the mood was in each case (for example: *Intense*). The computer looks at all these examples and learns the patterns.

Once it has learned, it can look at **new situations it has never seen before** and make a prediction: *"Based on what I learned, this zone is probably Energetic right now."*

This type of program is called a **classifier** — it classifies situations into categories. Our five categories are: **Energetic, Intense, Busy, Relaxed, and Calm**.

What is a Random Forest?

The technique we use is called a **Random Forest**. Here is how to think about it:

Imagine you are trying to decide where to go out tonight. Instead of asking one friend, you ask **100 different friends**, each of whom pays attention to different things — one focuses on the weather, another on whether PSV is playing, another on how many events are listed. Each friend gives you their opinion, and in the end you go with the **majority vote**.

That is exactly what a Random Forest does. It builds 100 small decision trees, each looking at the data slightly differently, and combines their votes into one final answer. This makes it much more reliable than asking just one.

How to use this notebook

You do not need to understand the code. Just follow these steps:

1. Click **Runtime** → **Run all** at the top of the page
2. When Step 2 asks you to upload a file, upload **mood_training_data_v1.csv**
3. Wait for all cells to finish running (a checkmark appears on the left of each one)
4. At the end, three charts and the trained model file will download automatically
5. Copy the text output from **Step 11** into your Word report

You only need to touch the keyboard once — at Step 2 when you upload the file. Everything else is automatic.

Step 1 — Install & Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import joblib
import warnings
warnings.filterwarnings("ignore")

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score, f1_score
from google.colab import files

print("All libraries loaded successfully")

All libraries loaded successfully
```

Step 2 — Upload the Training CSV

Run this cell and upload mood_training_data_v1.csv when the file picker appears.

```
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(f"Loaded: {filename}")
print(f"Rows: {len(df)}")
df.head()
```

<IPython.core.display.HTML object>

Saving mood_training_data_v1.csv to mood_training_data_v1.csv
Loaded: mood_training_data_v1.csv
Rows: 50

```
{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 50,\n  \"fields\": [\n    {\n      \"column\": \"date\",\n      \"properties\": {\n        \"dtype\": \"object\",\n        \"num_unique_values\": 44,\n        \"samples\": [\n          \"2024-01-27\",\n          \"2024-03-16\",\n          \"2024-12-31\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"hour\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 4,\n        \"min\": 7,\n        \"max\": 23,\n        \"num_unique_values\": 15,\n        \"samples\": [\n          16,\n
```

```

23,\n          20\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n            \"column\":\n            \"day_of_week\",\n            \"properties\": {\n              \"dtype\":\n              \"number\",\n              \"std\": 1,\n              \"min\": 0,\n              \"max\": 6,\n              \"num_unique_values\": 6,\n              \"samples\":\n              [\n                5,\n                1,\n                6\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"zone_id\",\n              \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\":\n                3,\n                \"samples\": [\n                  \"centrum\",\n                  \"strijs_s\",\n                  \"woensel_noord\"\n                ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"event_count\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\":\n                1,\n                \"min\": 0,\n                \"max\": 6,\n                \"num_unique_values\": 7,\n                \"samples\": [\n                  4,\n                  3,\n                  1\n                ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"event_category\",\n              \"properties\": {\n                \"dtype\":\n                \"category\",\n                \"num_unique_values\": 5,\n                \"samples\":\n                [\n                  \"none\",\n                  \"culture\",\n                  \"market\"\n                ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"psv_home_match\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\":\n                0,\n                \"min\": 0,\n                \"max\": 1,\n                \"num_unique_values\": 2,\n                \"samples\": [\n                  1,\n                  0\n                ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"temperature_c\",\n              \"properties\": {\n                \"dtype\":\n                \"number\",\n                \"std\": 6.778747099936533,\n                \"min\":\n                3.2,\n                \"max\": 28.4,\n                \"num_unique_values\": 44,\n                \"samples\": [\n                  4.3,\n                  28.4\n                ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"precipitation_mm\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\":\n                0.8286428540643046,\n                \"min\": 0.0,\n                \"max\": 4.2,\n                \"num_unique_values\": 7,\n                \"samples\": [\n                  0.0,\n                  1.2\n                ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"public_holiday\",\n              \"properties\": {\n                \"dtype\":\n                \"number\",\n                \"std\": 0,\n                \"min\": 0,\n                \"max\": 1,\n                \"num_unique_values\": 2,\n                \"samples\":\n                [\n                  1,\n                  0\n                ],\n              \"semantic_type\":\n              \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"mood_label\",\n              \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\": 5,\n                \"samples\": [\n                  \"Calm\",\n                  \"Relaxed\"\n                ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\":

```

```

\"notes\", \n      \"properties\": { \n          \"dtype\": \"string\", \n          \"num_unique_values\": 50, \n          \"samples\": [ \n              \"GLOW festival Strijp-S installations drawing large crowds\", \n              \"Sunday morning residential zone no events light rain quiet\" \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n      } \n  ], \n  \"type\": \"dataframe\", \"variable_name\": \"df\"}

```

Step 3 — Explore the Dataset

Copy these outputs into the Dataset Overview section of your report.

```

print("=" * 50)
print("DATASET OVERVIEW")
print("=" * 50)
print(f"Total training examples : {len(df)}")
print(f"Mood classes           : {df['mood_label'].nunique()}")
print()
print("Label distribution:")
label_counts = df["mood_label"].value_counts()
for label, count in label_counts.items():
    bar = chr(9608) * count
    print(f"  {label:<12} {count:>3}  {bar}")
print()
print("Zone distribution:")
print(df["zone_id"].value_counts().to_string())

```

```

=====
DATASET OVERVIEW
=====

Total training examples : 50
Mood classes           : 5

Label distribution:
Energetic      13  ██████████
Busy           11  ████████
Relaxed        10  ████████
Calm           9   ████████
Intense        7   ████████

Zone distribution:
zone_id
centrum      36
strijp_s     10
woensel_noord 4

MOOD_COLORS = {
    "Energetic": "#E67E22",
    "Intense":   "#C0392B",
    "Busy":      "#D35400",

```

```

    "Relaxed":    "#2471A3",
    "Calm":      "#1A5276"
}

fig, ax = plt.subplots(figsize=(8, 4))
labels = label_counts.index.tolist()
counts = label_counts.values.tolist()
colors = [MOOD_COLORS.get(l, "#888") for l in labels]
bars = ax.bar(labels, counts, color=colors, edgecolor="white",
linewidth=1.5)
ax.set_title("Training Label Distribution", fontsize=14,
fontweight="bold", pad=15)
ax.set_xlabel("Mood Label", fontsize=11)
ax.set_ylabel("Number of Examples", fontsize=11)
ax.set_ylim(0, max(counts) + 3)
for bar, count in zip(bars, counts):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
            str(count), ha="center", va="bottom", fontweight="bold")
plt.tight_layout()
plt.savefig("label_distribution.png", dpi=150, bbox_inches="tight")
plt.show()
print("Chart saved as label_distribution.png")

```

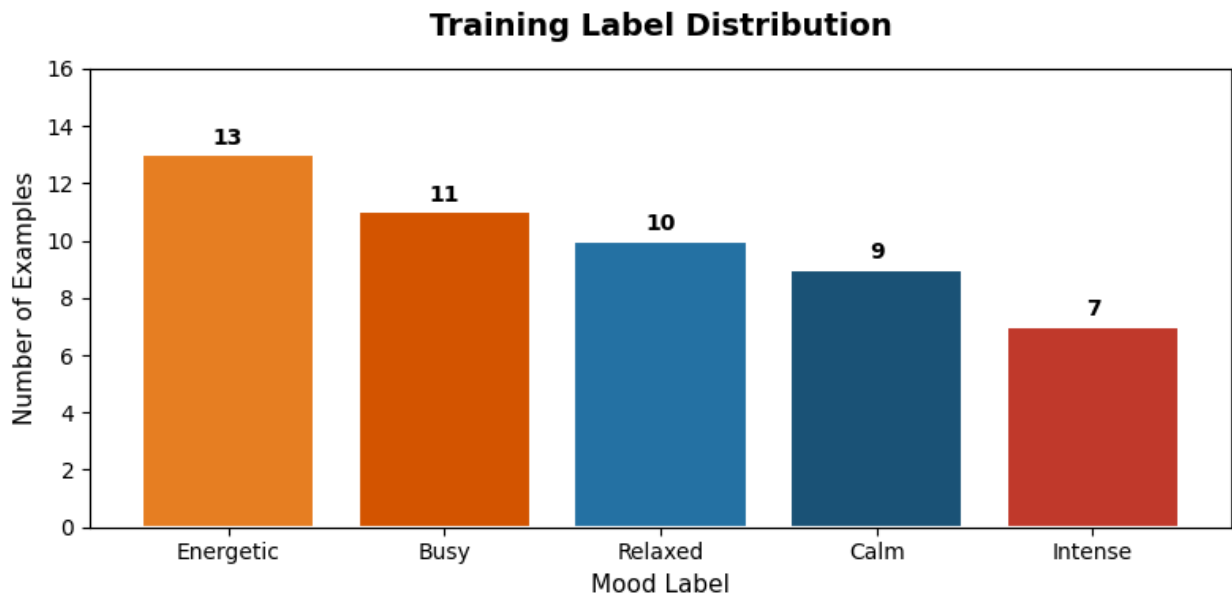


Chart saved as label_distribution.png

Step 4 — Feature Engineering

```

category_map = {"none": 0, "music": 1, "sport": 2, "culture": 3,
"market": 4}
df["event_category_encoded"] = df["event_category"].map(category_map)

```

```

FEATURES = [
    "event_count",
    "event_category_encoded",
    "psv_home_match",
    "hour",
    "day_of_week",
    "temperature_c",
    "precipitation_mm",
    "public_holiday"
]
TARGET = "mood_label"
X = df[FEATURES]
y = df[TARGET]
print("Feature matrix shape:", X.shape)
print("Target classes:", sorted(y.unique()))
X.describe().round(2)

```

```

Feature matrix shape: (50, 8)
Target classes: ['Busy', 'Calm', 'Energetic', 'Intense', 'Relaxed']

```

```

{"summary": "{\n  \"name\": \"X\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"event_count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 16.930905331881778,\n        \"min\": 0.0,\n        \"max\": 50.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          2.32,\n          2.0,\n          50.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"event_category_encoded\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 17.037840186311342,\n        \"min\": 0.0,\n        \"max\": 50.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          1.94,\n          1.5,\n          50.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"psv_home_match\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 17.603753201681577,\n        \"min\": 0.0,\n        \"max\": 50.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.16,\n          1.0,\n          0.37\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"hour\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 13.983981077851695,\n        \"min\": 4.19,\n        \"max\": 50.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          17.2,\n          19.0,\n          50.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"day_of_week\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 16.45690128790958,\n        \"min\": 0.0,\n        \"max\": 50.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          50.0,\n          4.72,\n          6.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  }\n}

```

```

}\n    },\n    {\n        \"column\": \"temperature_c\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 15.209270434273208, \n            \"min\": 3.2, \n            \"max\": 50.0, \n            \"num_unique_values\": 8, \n            \"samples\": [\n                13.8, \n                14.0, \n                50.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }, \n        {\n            \"column\": \"precipitation_mm\", \n            \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 17.468370549162767, \n                \"min\": 0.0, \n                \"max\": 50.0, \n                \"num_unique_values\": 5, \n                \"samples\": [\n                    0.28, \n                    4.2, \n                    0.83\n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\"\n            }, \n            {\n                \"column\": \"public_holiday\", \n                \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 17.61533402627219, \n                    \"min\": 0.0, \n                    \"max\": 50.0, \n                    \"num_unique_values\": 5, \n                    \"samples\": [\n                        0.06, \n                        1.0, \n                        0.24\n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\"\n                }, \n                }\n            }\n        ], \n        \"type\": \"dataframe\"}

```

Step 5 — Train / Test Split

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print(f\"Training set : {len(X_train)} examples\")
print(f\"Test set      : {len(X_test)} examples\")
print()
print(\"Class distribution in training set:\")
print(y_train.value_counts().to_string())

```

```

Training set : 40 examples
Test set      : 10 examples

```

```

Class distribution in training set:
mood_label
Energetic    10
Busy         9
Relaxed      8
Calm         7
Intense      6

```

Step 6 — Train the Random Forest Model

```

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=8,
    min_samples_leaf=2,
    random_state=42,
    class_weight=\"balanced\"
)

```

```

model.fit(X_train, y_train)
print("Model trained successfully")
print(f"Trees      : {model.n_estimators}")
print(f"Features   : {model.n_features_in_}")
print(f"Classes    : {list(model.classes_)}")

Model trained successfully
Trees      : 100
Features   : 8
Classes    : ['Busy', 'Calm', 'Energetic', 'Intense', 'Relaxed']

```

Step 7 — Evaluate the Model

Copy all output from this section into your report.

```

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
f1_weighted = f1_score(y_test, y_pred, average="weighted")

print("=" * 55)
print("MODEL EVALUATION RESULTS")
print("=" * 55)
print(f"Overall Accuracy   : {accuracy:.1%}")
print(f"Weighted F1-Score  : {f1_weighted:.3f}")
print()
print("Per-Class Report:")
print(classification_report(y_test, y_pred, zero_division=0))

```

```

=====
MODEL EVALUATION RESULTS
=====
Overall Accuracy   : 80.0%
Weighted F1-Score  : 0.725

Per-Class Report:

```

	precision	recall	f1-score	support
Busy	1.00	1.00	1.00	2
Calm	1.00	1.00	1.00	2
Energetic	0.60	1.00	0.75	3
Intense	1.00	1.00	1.00	1
Relaxed	0.00	0.00	0.00	2
accuracy			0.80	10
macro avg	0.72	0.80	0.75	10
weighted avg	0.68	0.80	0.72	10

```

cv_scores = cross_val_score(model, X, y, cv=5, scoring="f1_weighted")
print("=" * 55)

```

```

print("5-FOLD CROSS VALIDATION")
print("=" * 55)
print(f"Scores per fold : {[f"{s:.3f}" for s in cv_scores]}")
print(f"Mean F1       : {cv_scores.mean():.3f}")
print(f"Std deviation  : {cv_scores.std():.3f}")

=====
5-FOLD CROSS VALIDATION
=====
Scores per fold : ['0.893', '0.725', '0.900', '1.000', '0.733']
Mean F1       : 0.850
Std deviation  : 0.106

class_labels = sorted(y.unique())
cm = confusion_matrix(y_test, y_pred, labels=class_labels)
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_labels, yticklabels=class_labels,
            linewidths=0.5, ax=ax)
ax.set_title("Confusion Matrix - Test Set", fontsize=13,
            fontweight="bold", pad=15)
ax.set_xlabel("Predicted Label", fontsize=11)
ax.set_ylabel("True Label", fontsize=11)
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150, bbox_inches="tight")
plt.show()
print("Chart saved as confusion_matrix.png")

```

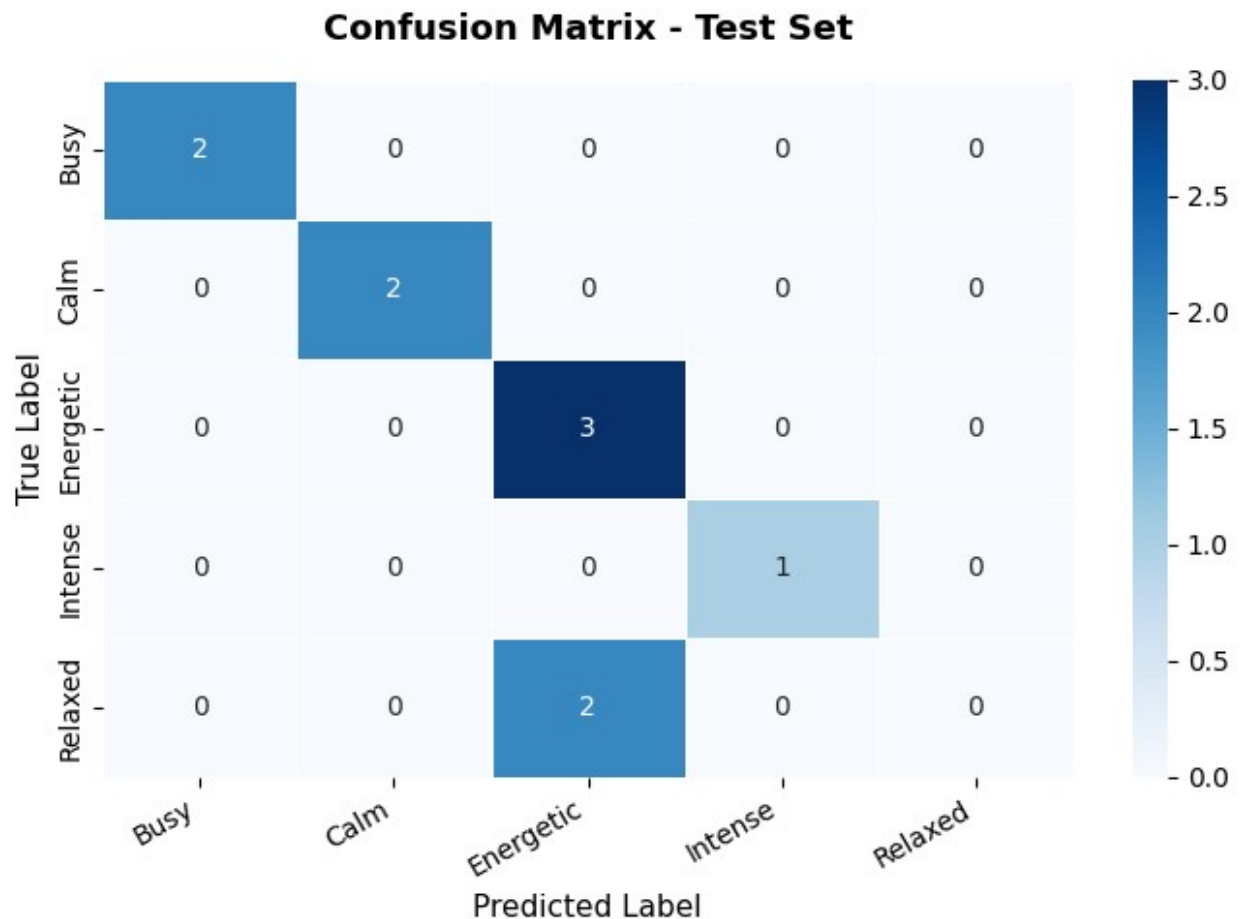


Chart saved as confusion_matrix.png

Step 8 — Feature Importance

```
importances = model.feature_importances_
importance_df = pd.DataFrame({
    "feature": FEATURES,
    "importance": importances
}).sort_values("importance", ascending=False)

print("=" * 50)
print("FEATURE IMPORTANCE RANKING")
print("=" * 50)
for _, row in importance_df.iterrows():
    bar = chr(9608) * int(row["importance"] * 40)
    print(f' {row["feature"]:<28} {row["importance"]:.3f} {bar} ')
print()
top2 = importance_df.head(2)
print(f'Top 2 features:')
print(f' 1. {top2.iloc[0]["feature"]} ({top2.iloc[0]
["importance"]:.1%})')
```

```
print(f' 2. {top2.iloc[1]["feature"]} ({top2.iloc[1]
["importance"]:.1%})')
```

```
=====
FEATURE IMPORTANCE RANKING
=====
```

event_category_encoded	0.279
psv_home_match	0.192
event_count	0.188
temperature_c	0.150
hour	0.132
precipitation_mm	0.036
day_of_week	0.016
public_holiday	0.006

Top 2 features:

1. event_category_encoded (27.9%)
2. psv_home_match (19.2%)

```
fig, ax = plt.subplots(figsize=(8, 4))
colors_fi = ["#1F4E79" if i < 2 else "#AED6F1" for i in
range(len(importance_df))]
bars = ax.barh(importance_df["feature"], importance_df["importance"],
               color=colors_fi, edgecolor="white")
ax.set_title("Feature Importance - Random Forest", fontsize=13,
fontweight="bold", pad=15)
ax.set_xlabel("Importance Score", fontsize=11)
ax.invert_yaxis()
for bar, val in zip(bars, importance_df["importance"]):
    ax.text(val + 0.002, bar.get_y() + bar.get_height()/2,
           f"{val:.3f}", va="center", fontsize=9)
legend_elements = [
    mpatches.Patch(color="#1F4E79", label="Top 2 features"),
    mpatches.Patch(color="#AED6F1", label="Other features")
]
ax.legend(handles=legend_elements, loc="lower right")
plt.tight_layout()
plt.savefig("feature_importance.png", dpi=150, bbox_inches="tight")
plt.show()
print("Chart saved as feature_importance.png")
```

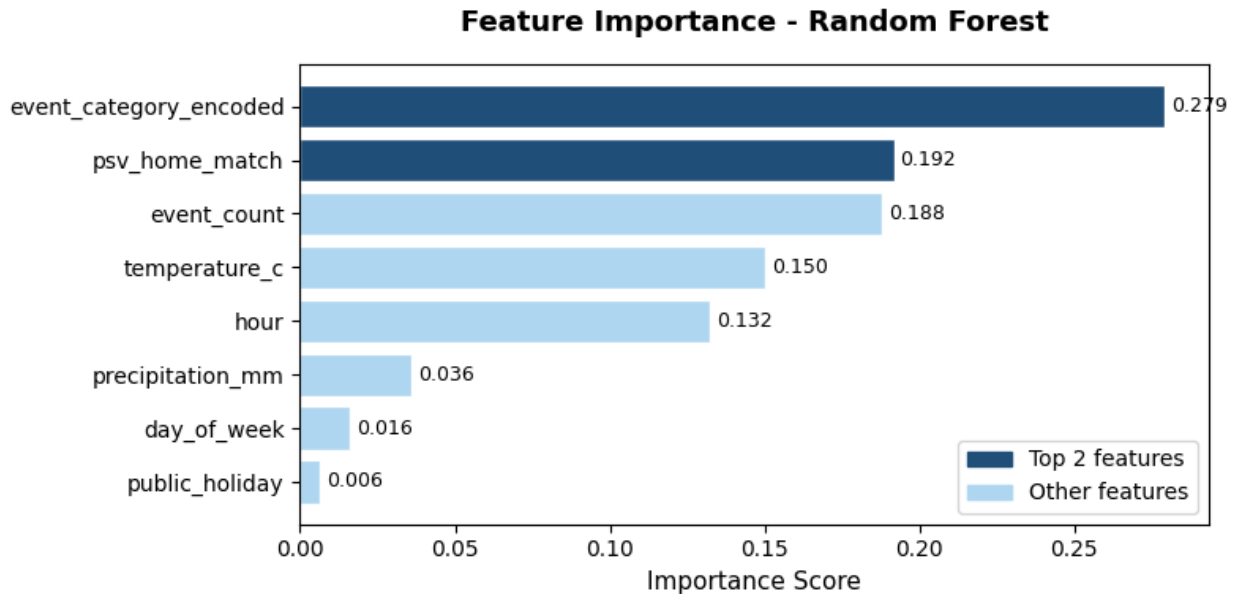


Chart saved as feature_importance.png

Step 9 — Test a Live Prediction

```
def predict_mood(event_count, event_category, psv_home_match,
                 hour, day_of_week, temperature_c,
                 precipitation_mm, public_holiday):
    cat_encoded = category_map.get(event_category, 0)
    features = [[event_count, cat_encoded, psv_home_match,
                 hour, day_of_week, temperature_c,
                 precipitation_mm, public_holiday]]
    label = model.predict(features)[0]
    proba = model.predict_proba(features)[0]
    confidence = max(proba)
    return label, confidence

scenarios = [
    (4, "music", 0, 21, 5, 18.5, 0.0, 0, "Saturday night, 4 music
events, warm"),
    (1, "sport", 1, 20, 5, 12.0, 0.0, 0, "PSV home match Saturday
evening"),
    (0, "none", 0, 7, 1, 6.0, 1.5, 0, "Tuesday early morning, cold
and rainy"),
]

print("=" * 60)
print("LIVE PREDICTION TEST")
print("=" * 60)
for s in scenarios:
    *args, desc = s
```

```

label, confidence = predict_mood(*args)
print(f"Scenario : {desc}")
print(f" Mood    : {label}")
print(f" Confidence: {confidence:.1%}")
print()

```

=====

LIVE PREDICTION TEST

=====

```

Scenario : Saturday night, 4 music events, warm
Mood      : Energetic
Confidence: 73.4%

```

```

Scenario : PSV home match Saturday evening
Mood      : Intense
Confidence: 79.7%

```

```

Scenario : Tuesday early morning, cold and rainy
Mood      : Calm
Confidence: 77.7%

```

Step 10 — Save & Download the Model

```

from datetime import date
model_filename = f"mood_model_v1_{date.today().strftime("%Y%m%d")}.pkl"
joblib.dump(model, model_filename)
print(f"Model saved as: {model_filename}")
files.download(model_filename)
files.download("label_distribution.png")
files.download("confusion_matrix.png")
files.download("feature_importance.png")
print("All files downloaded")

```

Model saved as: mood_model_v1_20260420.pkl

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

All files downloaded

Step 11 — Report Summary Block

Run this last. Copy the entire output into your Word report.

```
print("=" * 60)
print("ML MODEL TRAINING REPORT - SUMMARY")
print("Eindhoven Mood Radar | Fontys ICT | Moonshot Group")
print("=" * 60)
print()
print("DATASET")
print(f"  Total examples           : {len(df)}")
print(f"  Training set               : {len(X_train)}")
print(f"  Test set                   : {len(X_test)}")
print(f"  Number of features         : {len(FEATURES)}")
print(f"  Number of mood classes     : {len(model.classes_)}")
print()
print("MODEL CONFIGURATION")
print(f"  Algorithm                   : Random Forest Classifier (scikit-learn)")
print(f"  Number of trees             : {model.n_estimators}")
print(f"  Max tree depth              : {model.max_depth}")
print(f"  Class weighting             : balanced")
print()
print("EVALUATION RESULTS")
print(f"  Test set accuracy           : {accuracy:.1%}")
print(f"  Weighted F1-score           : {f1_weighted:.3f}")
print(f"  5-fold CV mean F1           : {cv_scores.mean():.3f} (+- {cv_scores.std():.3f})")
print()
print("FEATURE IMPORTANCE (ranked)")
for i, (_, row) in enumerate(importance_df.iterrows(), 1):
    print(f"  {i}. {row['feature']:<28} {row['importance']:.3f}")
print()
print("KNOWN LIMITATIONS")
print("  - Training set is manually labelled by the team, not residents")
print("  - 50 examples is small; accuracy will improve with more data")
print("  - Non-commercial events are invisible to the model")
print("  - Seasonal drift not yet tested")
print("  - Woensel-Noord underrepresented in training data")
print("=" * 60)
```

```
=====
ML MODEL TRAINING REPORT - SUMMARY
```

DATASET

Total examples	: 50
Training set	: 40
Test set	: 10
Number of features	: 8
Number of mood classes	: 5

MODEL CONFIGURATION

Algorithm	: Random Forest Classifier (scikit-learn)
Number of trees	: 100
Max tree depth	: 8
Class weighting	: balanced

EVALUATION RESULTS

Test set accuracy	: 80.0%
Weighted F1-score	: 0.725
5-fold CV mean F1	: 0.850 (+-0.106)

FEATURE IMPORTANCE (ranked)

1. event_category_encoded	0.279
2. psv_home_match	0.192
3. event_count	0.188
4. temperature_c	0.150
5. hour	0.132
6. precipitation_mm	0.036
7. day_of_week	0.016
8. public_holiday	0.006

KNOWN LIMITATIONS

- Training set is manually labelled by the team, not residents
 - 50 examples is small; accuracy will improve with more data
 - Non-commercial events are invisible to the model
 - Seasonal drift not yet tested
 - Woensel-Noord underrepresented in training data
- =====